Labshhet4

Task 1 : File name- Sum.js

```
function sum(a, b) {
   return a + b;  }
module.exports = sum;
```

even-odd.js

```
function checkEvenOdd(num) {
   return num % 2 === 0 ? "Even" : "Odd";
}
module.exports = checkEvenOdd;
```

```
Sum of 5 and 10 is: 15
Number 7 is: Odd
```

app1.js

```
const sum = require('./sum');
const checkEvenOdd = require('./even-odd');
console.log("Sum of 5 and 10 is:", sum(5, 10));
console.log("Number 7 is:", checkEvenOdd(7));
```

Task 2 : File name- file-ops.js

```
const fs = require('fs');
fs.writeFile('student.txt', 'Name: Pragya Singhal\nRoll No: 101\n', (err) => {
   if (err) throw err;
   console.log('File created and initial data written.');
   fs.appendFile('student.txt', 'Course: B.Tech Computer Science\n', (err) => {
      if (err) throw err;
      console.log('Course appended.');
      fs.readFile('student.txt', 'utf8', (err, data) => {
         if (err) throw err;
         console.log('\n--- File Content ---');
         console.log(data);
         fs.rename('student.txt', 'profile.txt', (err) => {
            if (err) throw err;
```

```
            console.log('File successfully renamed to profile.txt');
        });   });   }); });
```

```
Name: Pragya Singhal
Roll No: 101
Course: B.Tech Computer Science
```

Task 3 : File name- dir-ops.js

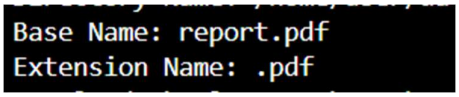
```
const fs = require('fs');
const path = require('path');
const dirName = './uploads';
fs.mkdir(dirName, { recursive: true }, (err) => {
   if (err) throw err;
   console.log('Directory "uploads" created.');
   fs.writeFile(path.join(dirName, 'file1.txt'), '', (err) => {
      fs.writeFile(path.join(dirName, 'file2.txt'), '', (err) => {
         fs.writeFile(path.join(dirName, 'file3.txt'), '', (err) => {
            console.log('3 empty files created inside uploads.');
            fs.readdir(dirName, (err, files) => {
               if (err) throw err;
               console.log('Files in folder:', files);
               fs.unlink(path.join(dirName, 'file3.txt'), (err) => {
                  if (err) throw err;
                  console.log('file3.txt deleted.');
                  fs.readdir(dirName, (err, remainingFiles) => {
                     console.log('Remaining files:', remainingFiles);
                  }); }); });  }); }); });});
```

```
Directory "uploads" created.
3 empty files created inside uploads.
Files in folder: [ 'file1.txt', 'file2.txt', 'file3.txt' ]
file3.txt deleted.
Remaining files: [ 'file1.txt', 'file2.txt' ]
```

Task 4 : File name- path-demo.js

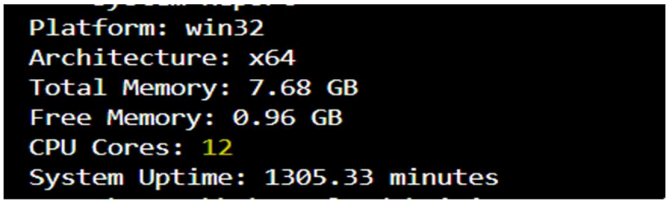
```
// path-demo.js
```

```
const path = require('path');
const filePath = '/home/user/data/report.pdf';
// Using path methods as specified
console.log('Directory Name:', path.dirname(filePath));
console.log('Base Name:', path.basename(filePath));
console.log('Extension Name:', path.extname(filePath));
// Absolute path resolved from a relative one using path.resolve
const relativePath = './data/report.pdf';
console.log('Resolved Absolute Path:', path.resolve(relativePath));
```



Task 5: File name- sys-report.js

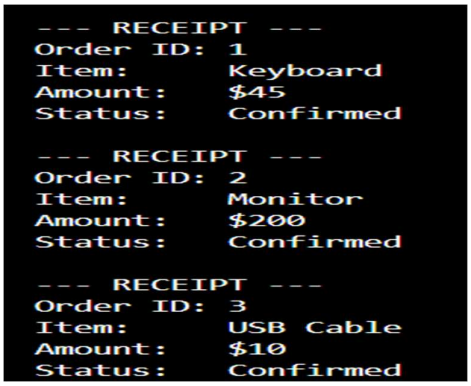
```
// sys-report.js
const os = require('os');
console.log('--- System Report ---');
console.log('Platform:', os.platform());
console.log('Architecture:', os.arch());
console.log('Total Memory:', (os.totalmem() / (1024 * 1024 * 1024)).toFixed(2), 'GB');
console.log('Free Memory:', (os.freemem() / (1024 * 1024 * 1024)).toFixed(2), 'GB');
console.log('CPU Cores:', os.cpus().length);
console.log('System Uptime:', (os.uptime() / 60).toFixed(2), 'minutes');
```



Task 6: File name – order-events.js

```
const EventEmitter = require('events').EventEmitter;
const orderEmitter = new EventEmitter();
orderEmitter.on('orderPlaced', (order) => {
    console.log('\n--- RECEIPT ---');
    console.log(`Order ID: ${order.id}`);
```

```
    console.log(`Item:     ${order.item}`);
    console.log(`Amount:   $${order.amount}`);
    console.log('Status:   Confirmed');
});
orderEmitter.emit('orderPlaced', { id: 1, item: 'Keyboard', amount: 45 });
orderEmitter.emit('orderPlaced', { id: 2, item: 'Monitor', amount: 200 });
orderEmitter.emit('orderPlaced', { id: 3, item: 'USB Cable', amount: 10 });
```



Task7 : File name- server.js

```
// server.js
const http = require('http');
const url = require('url');
const students = [
    { id: 1, name: 'Alice' },
    { id: 2, name: 'Bob' },
    { id: 3, name: 'Charlie' }
];
const server = http.createServer((req, res) => {
    const parsedUrl = url.parse(req.url, true);
    const pathname = parsedUrl.pathname;
    if (req.method === 'GET' && pathname === '/') {
        res.writeHead(200, { 'Content-Type': 'text/plain' });
        res.end('Welcome to the homepage!');
    }
    else if (req.method === 'GET' && pathname === '/students') {
```

```
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify(students));
  }
  else if (req.method === 'GET' && pathname.startsWith('/students/')) {
    const id = parseInt(pathname.split('/')[2]);
    const student = students.find(s => s.id === id);

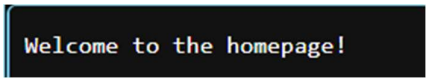
    if (student) {
      res.writeHead(200, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify(student));
    } else {
      res.writeHead(404, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ error: 'Student not found (404)' }));
    } }
  else {
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.end('404 Page Not Found');
  } });
server.listen(4000, () => {
  console.log('Server running on http://localhost:4000');  });
```

```
Welcome to the homepage!
```

Task 8: File name – query-demo.js

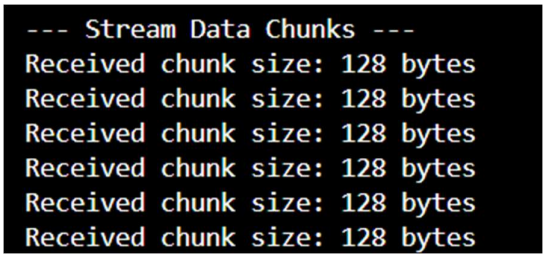
```
const http = require('http');
const url = require('url');
const server = http.createServer((req, res) => {
  const parsedUrl = url.parse(req.url, true);
  const pathname = parsedUrl.pathname;
  if (req.method === 'GET' && pathname === '/search') {
    const keyword = parsedUrl.query.keyword;
    res.writeHead(200, { 'Content-Type': 'application/json' });
```

```
        res.end(JSON.stringify({
            message: 'Keyword successfully parsed',
            keyword: keyword || 'No keyword provided'
        }));
    } else {
        res.writeHead(404, { 'Content-Type': 'text/plain' });
        res.end('404 Not Found. Please use the /search route.');
    } });
server.listen(4001, () => {
    console.log('Query Demo Server running on http://localhost:4000');
});
```



Task 9: File name- stream-demo.js

```
const fs = require('fs');
let lines = '';
for (let i = 1; i <= 55; i++) {
    lines += `This is line number ${i} generated for stream testing.\n`;
}
fs.writeFileSync('stream-file.txt', lines);
console.log('stream-file.txt created with 55 lines.');
const stream = fs.createReadStream('stream-file.txt', { encoding: 'utf8', highWaterMark: 128 });
console.log('\n--- Stream Data Chunks ---');
stream.on('data', (chunk) => {
    console.log(`Received chunk size: ${chunk.length} bytes`);
});
stream.on('end', () => {
    console.log('--- Stream reading ended ---');
});
```

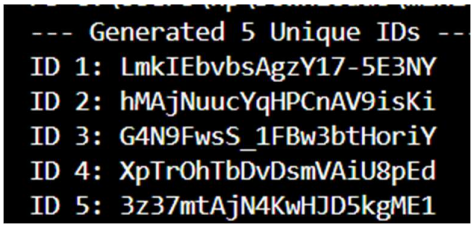


Task 10: File name- id-gen.js

```
const { nanoid } = require('nanoid');

console.log('--- Generated 5 Unique IDs ---');

for (let i = 1; i <= 5; i++) {
   console.log(`ID ${i}: ${nanoid()}`);
}
```



Task 11: File name- calc-cli.js

```
const args = process.argv.slice(2);

const num1 = parseFloat(args[0]);

const operator = args[1];

const num2 = parseFloat(args[2]);

if (isNaN(num1) || isNaN(num2)) {
   console.log('Please provide numbers. Example: node calc-cli.js 10 5 +');
   process.exit(1);
}

let result;

switch (operator) {
   case '+':
      result = num1 + num2;
      break;
   case '-':
      result = num1 - num2;
```

```
        break;
    case '*':
        result = num1 * num2;
        break;
    case '/':
        result = num2 !== 0 ? num1 / num2 : 'Error: Division by zero';
        break;
    default:
        result = 'Invalid operator. Use +, -, *, /';
}
console.log('Result:', result);
```

```
PS C:\Users\hp\Downloads\mini-ecommerce-frontend-main (1)\mini-ecommerce-frontend-main\labsheet4> node calc-cli.js 10 "+" 5
Result: 15
PS C:\Users\hp\Downloads\mini-ecommerce-frontend-main (1)\mini-ecommerce-frontend-main\labsheet4> node calc-cli.js 20 / 4
Result: 5
```

Task 12: File name- env-demo.js

```
const http = require('http');
const PORT = process.env.PORT || 4000;
const server = http.createServer((req, res) => {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end(`Server running on custom environment port: ${PORT}`);
});
server.listen(PORT, () => {
    console.log(`Active Port: ${PORT}`);
});
```